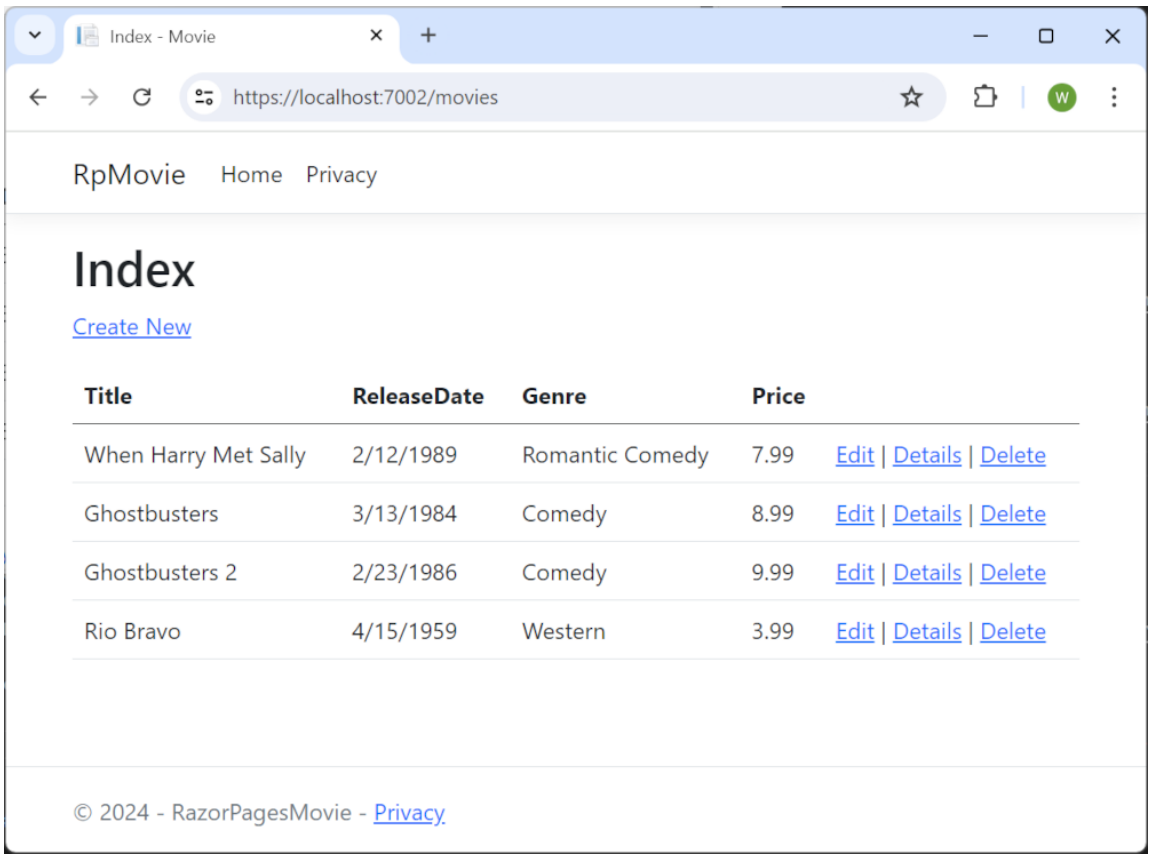
Початок роботи з C# і ASP.NET Core в Visual Studio. Razor Pages (частина 2)

Завдання. Продовжіть створення додатку Razor Pages за прикладом нижче наведеного Каталогу фільмів.

Замість каталогу фільмів, що наводиться в прикладі зберігайте каталог книг, музики, товарів, адрес і т.д. на власний вибір. **Ваша програма не повинна бути повністю ідентичною до опису в прикладі.**

Оновлення згенерованих сторінок у додатку ASP.NET Core

Додаток для перегляду фільмів має хороший початок, але він все ще не ідеальний. **ReleaseDate** має складатися з двох слів: **Release Date** (або українською «Дата виходу»)



Оновити модель

Оновіть `models/Movie.cs` за допомогою наступного виділеного коду:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace RazorPagesMovie.Models;

public class Movie
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;

 [Display(Name = "Release Date")]
 [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }
    public string Genre { get; set; } = string.Empty;

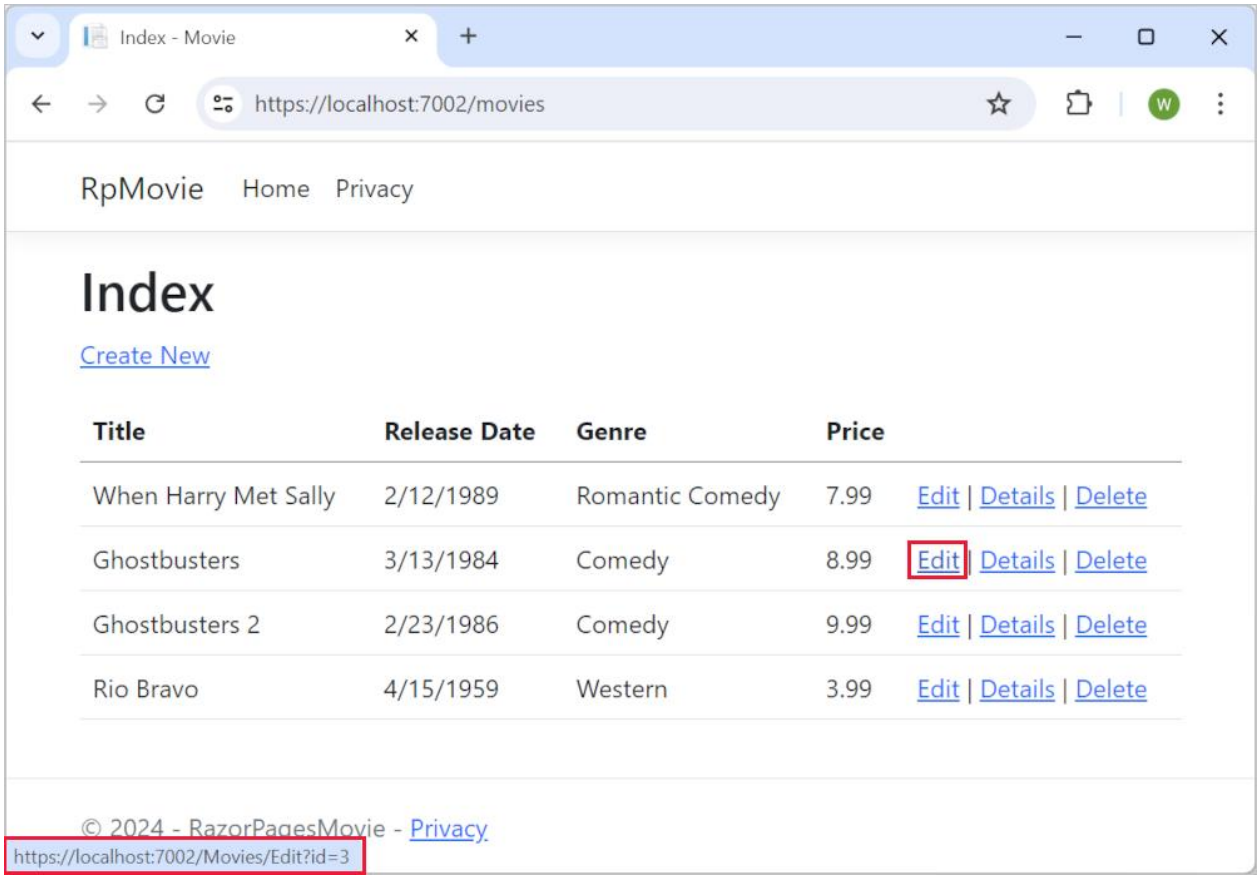
 [Column(TypeName = "decimal(18, 2)")]
    public decimal Price {get; set;}
}
```

У попередньому коді:

- Анотація даних `[Column(TypeName = "decimal(18, 2)")]` дозволяє Entity Framework Core правильно зіставляти `ціну` з валютою в базі даних. Щоб отримати докладнішу інформацію, перегляньте розділ Типи даних.
- Атрибут [Display] визначає коротке ім'я поля. У попередньому коді `Release Date` замість `ReleaseDate`.
- Атрибут [DataType] визначає тип даних (`Date`). Інформація про час, що зберігається в полі, не відображається.

DataAnnotations розглядається пізніше

Перейдіть до *розділу Pages/Movies* та наведіть курсор на посилання Edit (**Редагувати)**, щоб побачити цільову URL-адресу.



Посилання **«Редагувати»**, **«Деталі»** та **«Видалити»** генеруються допоміжним тегом прив'язки у файлі `Pages/Movies/Index.cshtml`.

CSHTML

```
@foreach (var item in Model.Movie) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Title)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.ReleaseDate)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Genre)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Price)
            </td>
            <td>
                <a asp-page="./Edit" asp-route-id="@item.Id">Edit</a> |
                <a asp-page="./Details" asp-route-id="@item.Id">Details</a> |
                <a asp-page="./Delete" asp-route-id="@item.Id">Delete</a>
            </td>
        </tr>
}
    </tbody>
</table>
```

Tag Helpers дозволяють коду на стороні сервера брати участь у створенні та відображенні елементів HTML у файлах Razor.

У попередньому коді Anchor Tag Helper динамічно генерує значення атрибута HTML `href` зі сторінки Razor (маршрут відносний), `asp-сторінки` та ідентифікатора маршруту (`asp-route-id`).

Використовуйте **View Source** у браузері, щоб перевірити згенеровану розмітку. Частина згенерованого HTML показана нижче:

Код HTML

```
<td>
  <a href="/Movies/Edit?id=1">Edit</a> |
  <a href="/Movies/Details?id=1">Details</a> |
  <a href="/Movies/Delete?id=1">Delete</a>
</td>
```

Динамічно згенеровані посилання передають ідентифікатор фільму з рядком запиту. Наприклад, ?id=1 в https://localhost:5001/Movies/Details?id=1.

Додавання шаблона маршруту

Оновіть сторінки Razor «Редагування», «Деталі» та «Видалити», щоб використовувати шаблон маршруту {id:int}. Змініть директиву сторінки для кожної з цих сторінок з @page на @page "{id:int}". Запустіть програму, а потім перегляньте джерело. Згенерований HTML додає ідентифікатор до частини шляху URL:

```
Код HTML
<td>
 <a href="/Movies/Edit/1">Edit</a> |
 <a href="/Movies/Details/1">Details</a> |
 <a href="/Movies/Delete/1">Delete</a>
</td>
```

Запит на сторінку з шаблоном маршруту {id:int}, який **не** включає ціле число, повертає помилку HTTP 404 (не знайдено). Наприклад, https://localhost:5001/Movies/Details повертає помилку 404. Щоб зробити ідентифікатор необов'язковим, додайте ? до обмеження маршруту:

CSHTML

```
@page "{id:int?}"
```

Перевірте поведінку @page "{id:int?}":

1. Встановіть директиву сторінки у Pages/Movies/Details.cshtml як @page "{id:int?}".
2. Встановіть точку зупинки в public async Task<IActionResult> OnGetAsync(int? id), в Pages/Movies/Details.cshtml.cs.
3. Перейдіть до https://localhost:5001/Movies/Details/

За допомогою директиви @page "{id:int}" точка зупинки ніколи не буде досягнута. Механізм маршрутизації повертає HTTP 404. Через використання @page "{id:int?}", метод OnGetAsync повертає NotFound (HTTP 404):

```
public async Task<IActionResult> OnGetAsync(int? id)
{
    if (id == null)
    {
        return NotFound();
    }
```

```
    var movie = await _context.Movie.FirstOrDefaultAsync(m => m.Id == id);
    if (movie == null)
    {
        return NotFound();
    }
    else
    {
        Movie = movie;
    }
    return Page();
}
```

Перегляд обробки винятків паралельності

Перегляньте метод OnPostAsync у файлі Pages/Movies/Edit.cshtml.cs:

```
public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _context.Attach(Movie).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!MovieExists(Movie.Id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }

    return RedirectToPage("./Index");
}

private bool MovieExists(int id)
{
    return _context.Movie.Any(e => e.Id == id);
}
```

Попередній код виявляє винятки паралелізму, коли один клієнт видаляє фільм, а інший – вносить зміни до фільму.

Щоб протестувати блок catch:

1. Встановіть точку зупинки на catch (DbUpdateConcurrencyException).
2. Виберіть **Редагувати** для фільму, внесіть зміни, але не вводьте **Зберегти**.
3. В іншому вікні браузера виберіть посилання **Видалити** для того самого фільму, а потім видаліть фільм.
4. У попередньому вікні браузера опублікуйте зміни у фільмі.

Огляд публікування та зв’язування

Перегляньте файл `Pages/Movies/Edit.cshtml.cs`:

```
public class EditModel : PageModel
{
    private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;

    public EditModel(RazorPagesMovie.Data.RazorPagesMovieContext context)
    {
        _context = context;
    }

    [BindProperty]
    public Movie Movie { get; set; } = default!;

    public async Task<IActionResult> OnGetAsync(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var movie =  await _context.Movie.FirstOrDefaultAsync(m => m.Id == id);
        if (movie == null)
        {
            return NotFound();
        }
        Movie = movie;
        return Page();
    }

// Щоб захиститися від атак перевантаження, увімкніть конкретні властивості, до яких
потрібно прив’язати дані.
// Для отримання додаткової інформації див. https://aka.ms/RazorPagesCRUD.
    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
        {
            return Page();
        }

        _context.Attach(Movie).State = EntityState.Modified;

        try
        {
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!MovieExists(Movie.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }

        return RedirectToPage("./Index");
    }

    private bool MovieExists(int id)
```

```
    {
        return _context.Movie.Any(e => e.Id == id);
    }
```

Наприклад, коли запит HTTP GET надходить на сторінку Movies/Edit, https://localhost:5001/Movies/Edit/3:

- Метод **OnGetAsync** отримує фільм із бази даних і повертає метод Page.
- Метод **Page** відображає сторінку Razor **Pages/Movies/Edit.cshtml**. Файл **Pages/Movies/Edit.cshtml** містить директиву моделі **@model RazorPagesMovie.Pages.Movies.EditModel**, яка робить модель фільму доступною на сторінці.
- Відобразиться форма Редагування зі значеннями з фільму.

Коли публікується сторінка Movies/Edit:

- Значення форми на сторінці прив'язані до властивості Movie. Атрибут [BindProperty] включає прив'язку моделі.

```
[BindProperty]
public Movie Movie { get; set; }
```

- Якщо в стані моделі є помилки, наприклад, ReleaseDate не вдається перетворити на дату, форма знову відображається з надісланими значеннями.
- Якщо помилок моделі немає, фільм зберігається.

Методи HTTP GET на сторінках Razor «Index», «Create» та «Delete» працюють за схожою схемою. Метод HTTP POST OnPostAsync на сторінці створення Razor працює за тією ж схемою, що й метод OnPostAsync на сторінці редагування Razor.

Додавання пошуку до ASP.NET основних сторінок Razor Pages

Додайте такий виділений код до Pages/Movies/Index.cshtml.cs:

```
public class IndexModel : PageModel
{
    private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;

    public IndexModel(RazorPagesMovie.Data.RazorPagesMovieContext context)
    {
        _context = context;
    }

    public IList<Movie> Movie { get;set; }  = default!;

    [BindProperty(SupportsGet = true)]
    public string? SearchString { get; set; }

    public SelectList? Genres { get; set; }

    [BindProperty(SupportsGet = true)]
```

```
    public string? MovieGenre { get; set; }
```

У попередньому коді:

SearchString: Містить текст, який користувачі вводять у поле пошуку. SearchString має атрибут [BindProperty]. [BindProperty] зв'язує значення з форми та рядки запиту з тією ж назвою, що й властивість. Для прив'язки під час HTTP GET-запитів потрібен атрибут [BindProperty(SupportsGet = true)].

Genres: Містить список жанрів. Genres дозволяє користувачеві вибрати жанр зі списку. Для використання SelectList необхідно підключити простір імен Microsoft.AspNetCore.Mvc.Rendering.

MovieGenre: Містить конкретний жанр, який обрав користувач. Наприклад, *«Western»*.

Genres і MovieGenre будуть використані пізніше у цьому навчальному посібнику.

Попередження

З міркувань безпеки необхідно явно дозволяти прив'язку даних GET-запитів до властивостей моделі сторінки. Перевіряйте введені користувачем дані перед тим, як зіставляти їх із властивостями. Дозвіл на прив'язку при GET-запитах корисний у сценаріях, що покладаються на рядки запиту або значення маршруту.

Щоб прив'язати властивість у GET-запитах, встановіть для атрибуту [BindProperty] параметр SupportsGet = true:

```
[BindProperty(SupportsGet = true)]
```

Оновіть метод OnGetAsync сторінки **Movies/Index** наступним кодом:

```
public async Task OnGetAsync()
{
    var movies = from m in _context.Movie
                 select m;
    if (!string.IsNullOrEmpty(SearchString))
    {
        movies = movies.Where(s => s.Title.Contains(SearchString));
    }

    Movie = await movies.ToListAsync();
}
```

Перший рядок методу `OnGetAsync` створює запит LINQ для вибору фільмів:

```
var movies = from m in _context.Movie
             select m;
```

Запит визначено лише на цьому етапі, він ***не*** був виконаний в базі даних. Якщо властивість `SearchString` не дорівнює `null` або порожня, запит movies змінюється для фільтра в рядку пошуку:

```
if (!string.IsNullOrEmpty(SearchString))
{
    movies = movies.Where(s => s.Title.Contains(SearchString));
}
```

Код s => s.Title.Contains() — це **лямбда-вираз**.
Лямбди використовуються в LINQ-запитах у методичному синтаксисі як аргументи стандартних методів операторів запиту, таких як Where або Contains.

LINQ-запити **не виконуються** у момент їх визначення чи зміни викликом методу (наприклад, Where, Contains або OrderBy). Натомість виконання запиту відкладається (*deferred execution*).
Обчислення виразу відкладається до того моменту, поки його фактичне значення не буде ітероване або поки не буде викликано метод ToListAsync.
Докладніше дивіться у розділі **Query Execution**.

Примітка

Метод Contains виконується у **базі даних**, а не у коді C#. Чутливість до регістру в запиті залежить від бази даних і від колації.

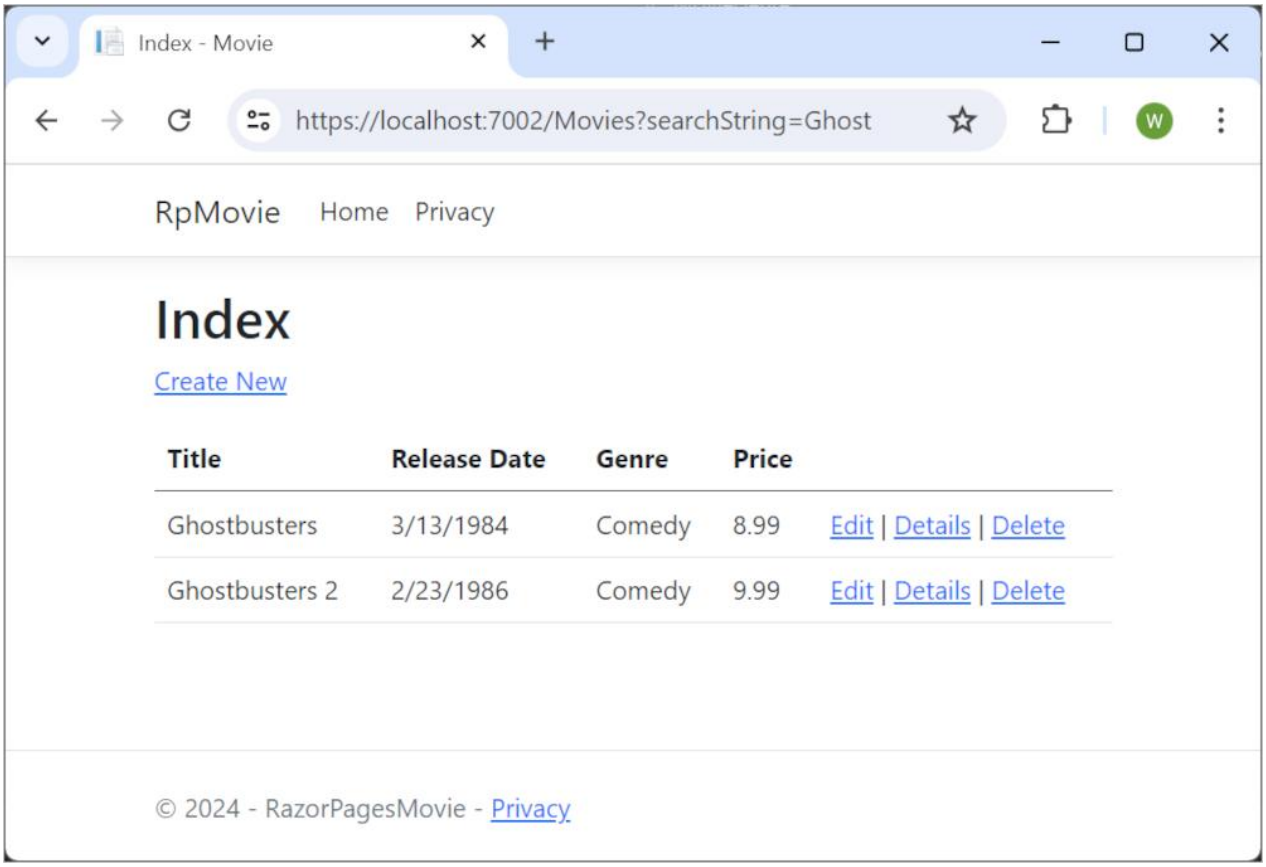
- У **SQL Server** Contains відображається на SQL LIKE, який не чутливий до регістру.
- У **SQLite** з типовою колацією чутливість до регістру є змішаною — частково чутливою, частково нечутливою залежно від запиту.

Інформацію про створення нечутливих до регістру запитів у SQLite можна знайти тут:

- How to use case-insensitive query with Sqlite provider? (dotnet/efcore #11414)
- How to make a SQLite column case insensitive (dotnet/AspNetCore.Docs #22314)
- Collations and Case Sensitivity

Перейдіть на сторінку **Movies** та додайте рядок запиту, наприклад **?searchString=Ghost**, до URL.

Наприклад:
https://localhost:5001/Movies?searchString=Ghost
Буде відображено відфільтровані фільми.

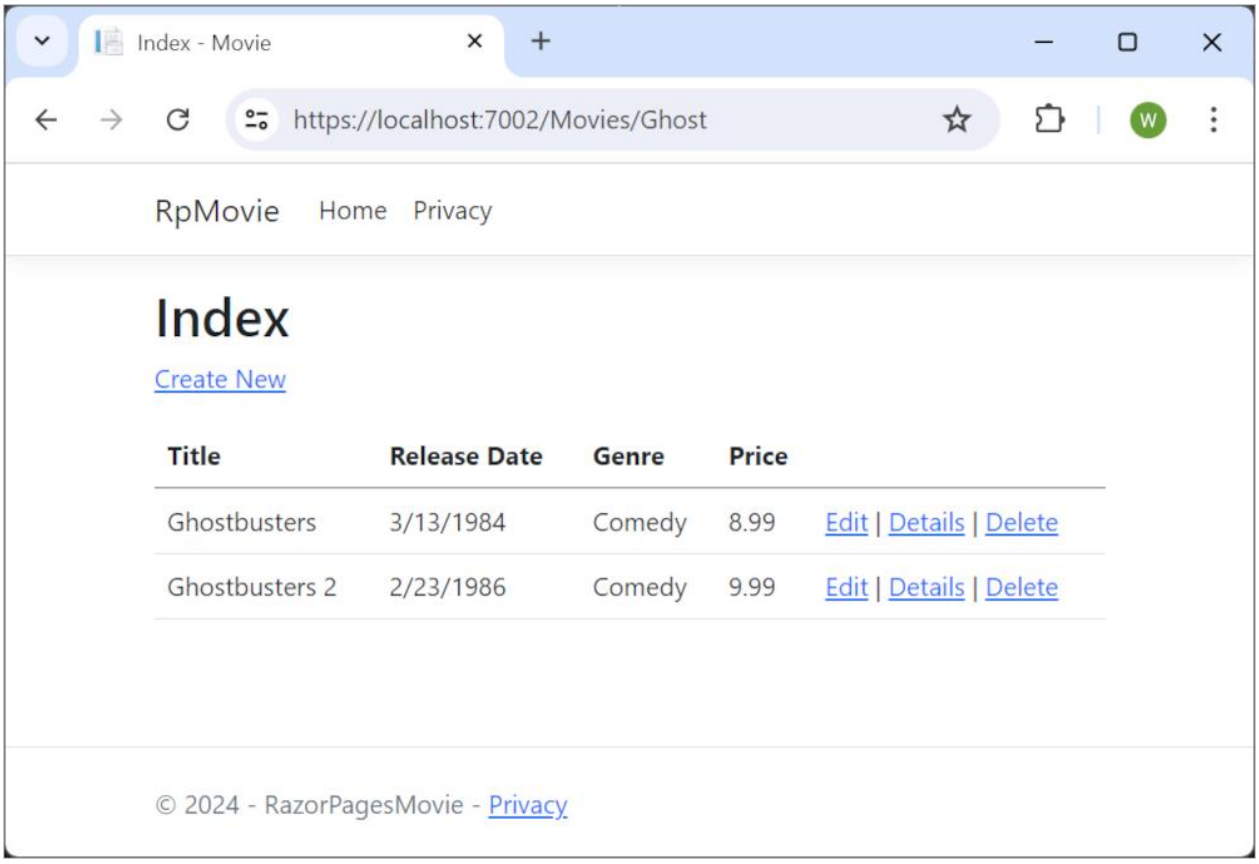


Якщо до сторінки **Index** додати наступний шаблон маршруту, то рядок пошуку можна буде передати як сегмент URL.
Наприклад:
https://localhost:5001/Movies/Ghost

CSHTML

```
@page "{searchString?}"
```

Наведене вище обмеження маршруту дозволяє шукати назву фільму як дані маршруту (сегмент URL), а не як значення рядка запиту.
Символ ? у "{searchString?}" означає, що цей параметр маршруту є **необов'язковим**.



Під час виконання ASP.NET Core використовує **зв'язування моделей** (*model binding*), щоб встановити значення властивості SearchString з рядка запиту (?searchString=Ghost) або з даних маршруту (https://localhost:5001/Movies/Ghost). Зв'язування моделей **не є чутливим до регістру**.

Однак, очікувати, що користувачі вручну змінюватимуть URL для пошуку фільму, недоцільно. На цьому етапі додається елемент **UI** для фільтрації фільмів. Якщо ви додали обмеження маршруту "{searchString?}", його потрібно **видалити**.

Відкрийте файл **Pages/Movies/Index.cshtml** і додайте розмітку, виділену в наведеному нижче коді:

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-page="Create">Create New</a>
</p>

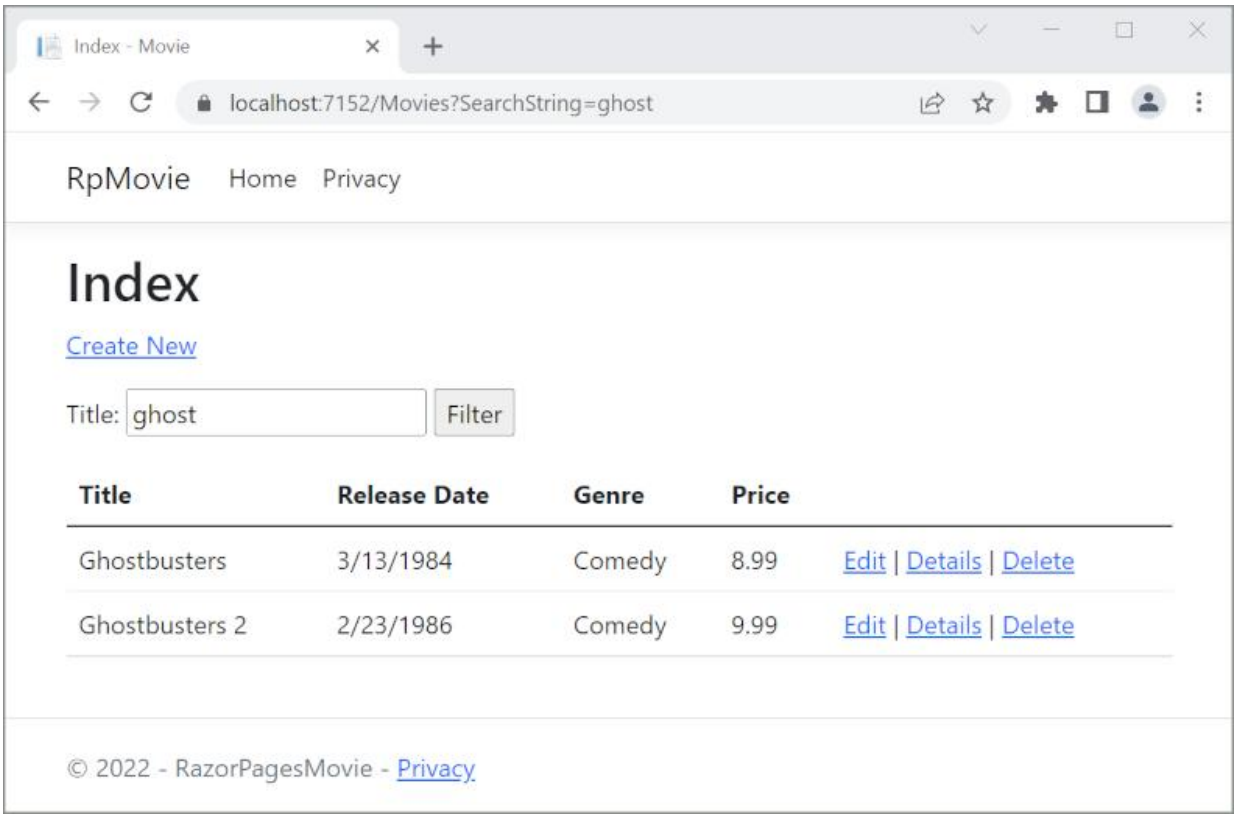
<form>
    <p>
        <label>Title: <input type="text" asp-for="SearchString" /></label>
        <input type="submit" value="Filter" />
    </p>
</form>

<table class="table">
    <thead>
```

Тег HTML `<form>` використовує такі допоміжні теги:

- Помічник тегів форми. Коли форма надсилається, рядок фільтра надсилається на сторінку *Pages/Movies/Index* за допомогою рядка запиту.
- Помічник тегів введення

Збережіть зміни та протестуйте фільтр.



Пошук за жанром

Оновіть метод **OnGetAsync** сторінки **Movies/Index.cshtml.cs** за допомогою наступного коду:

```
public async Task OnGetAsync()
{
    // <snippet_search_linqQuery>
    IQueryable<string> genreQuery = from m in _context.Movie
                                    orderby m.Genre
                                    select m.Genre;
    // </snippet_search_linqQuery>

    var movies = from m in _context.Movie
                 select m;

    if (!string.IsNullOrEmpty(SearchString))
    {
        movies = movies.Where(s => s.Title.Contains(SearchString));
    }

    if (!string.IsNullOrEmpty(MovieGenre))
    {
        movies = movies.Where(x => x.Genre == MovieGenre);
    }
```

```
    // <snippet_search_selectList>
    Genres = new SelectList(await genreQuery.Distinct().ToListAsync());
    // </snippet_search_selectList>
    Movie = await movies.ToListAsync();
}
```

Наступний код є запитом LINQ, який отримує всі жанри з бази даних.

```
IQueryable<string> genreQuery = from m in _context.Movie
                                orderby m.Genre
                                select m.Genre;
```

Список жанрів створюється шляхом проектування різних жанрів:

```
Жанри = новий SelectList(чекайте genreQuery.Distinct(). ToListAsync());
```

Додавання пошуку за жанром на сторінку Razor Page

Оновіть елемент <form> у Index.cshtml , як виділено в наступній розмітці:

CSHTML

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

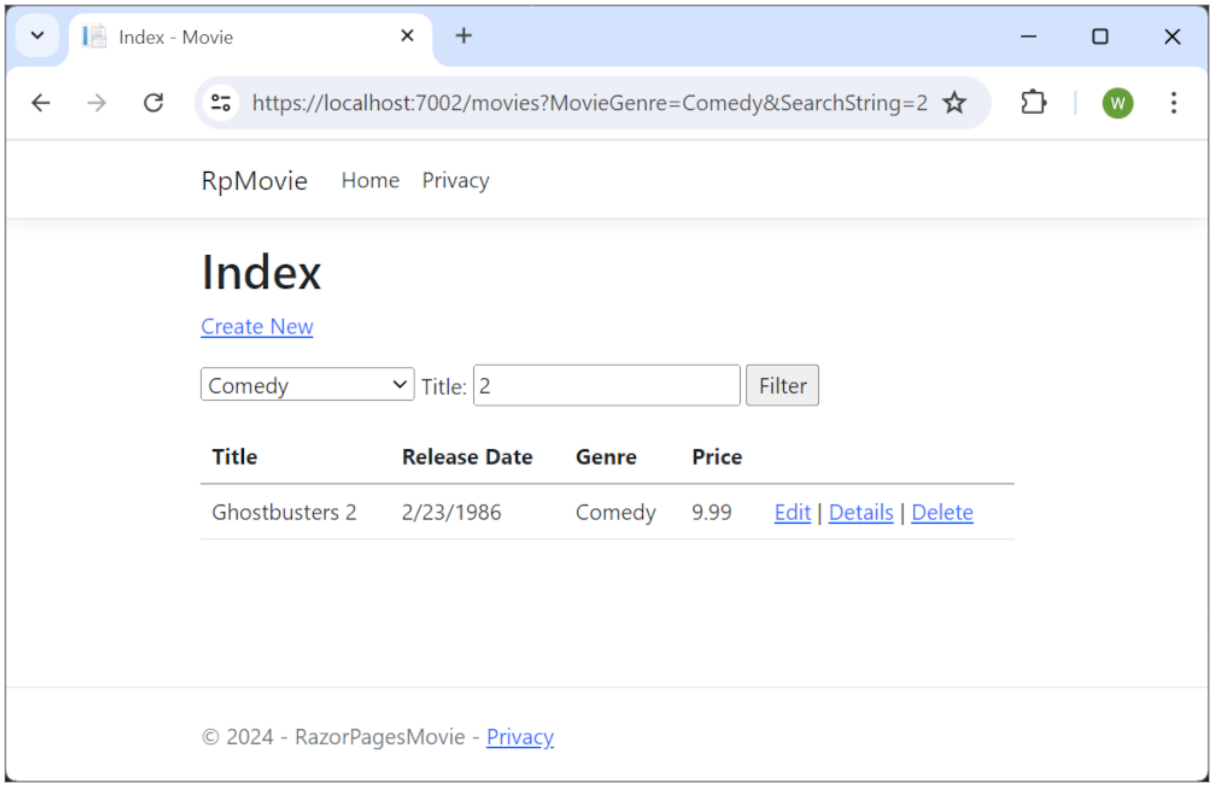
@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-page="Create">Create New</a>
</p>

<form>
    <p>
        <select asp-for="MovieGenre" asp-items="Model.Genres">
            <option value="">All</option>
        </select>
        <label>Title: <input type="text" asp-for="SearchString" /></label>
        <input type="submit" value="Filter" />
    </p>
</form>
```

Перевірте програму, виконавши пошук за жанром, назвою фільму та обома способами:



Додавання нового поля до сторінки Razor у ASP.NET Core

У цьому розділі **Entity Framework Core (EF Core)** використовується для визначення схеми бази даних на основі модельного класу застосунку:

- Додається нове поле до моделі.
- Здійснюється міграція зміни схеми (нового поля) до бази даних.

Підхід EF Core забезпечує більш гнучкий процес розробки. Розробник працює безпосередньо з моделлю даних застосунку, тоді як схема бази даних створюється та синхронізується автоматично, без необхідності перемикатися між кодом і інструментами керування базами даних. Огляд **Entity Framework Core** та його переваг дивіться у матеріалі *Entity Framework Core*.

Використання EF Core для автоматичного створення й відстеження бази даних:

- Додає таблицю __EFMigrationsHistory до бази даних для відстеження того, чи синхронізована схема бази даних із класами моделі, з яких вона була згенерована.
- Генерує виняток (*exception*), якщо класи моделі не синхронізовані з базою даних.

Автоматична перевірка синхронності схеми й моделі полегшує виявлення проблем із неконсистентним кодом бази даних.

Додавання властивості *Rating* до моделі *Movie*

1. Відкрийте файл **Models/Movie.cs** і додайте властивість Rating:

```
public class Movie
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;

    [Display(Name = "Release Date")]
    [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }
    public string Genre { get; set; } = string.Empty;

    [Column(TypeName = "decimal(18, 2)")]
    public decimal Price { get; set; }
    public string Rating { get; set; } = string.Empty;
}
```

2. Відредагуйте `Pages/Movies/Index.cshtml` і додайте поле `Rating`:

CSHTML

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-page="Create">Create New</a>
</p>

<form>
    <p>
        <select asp-for="MovieGenre" asp-items="Model.Genres">
            <option value="">All</option>
        </select>
        <label>Title: <input type="text" asp-for="SearchString" /></label>
        <input type="submit" value="Filter" />
    </p>
</form>

<table class="table">

    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Title)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].ReleaseDate)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Genre)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Price)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Rating)
            </th>
            <th></th>
        </tr>
    </thead>
    <tbody>
```

```
        @foreach (var item in Model.Movie)
        {
            <tr>
                <td>
                    @Html.DisplayFor(modelItem => item.Title)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.ReleaseDate)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Genre)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Price)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Rating)
                </td>
                <td>
                    <a asp-page="./Edit" asp-route-id="@item.Id">Edit</a> |
                    <a asp-page="./Details" asp-route-id="@item.Id">Details</a> |
                    <a asp-page="./Delete" asp-route-id="@item.Id">Delete</a>
                </td>
            </tr>
        }
    </tbody>
</table>
```

Оновіть наступні сторінки за допомогою поля `Rating`:

- Pages/Movies/Create.cshtml
- Pages/Movies/Delete.cshtml
- Pages/Movies/Details.cshtml
- Pages/Movies/Edit.cshtml

Додаток не працюватиме, доки база даних не буде оновлена для включення нового поля.
Запуск застосунку без оновлення бази даних призведе до винятку **SqlException**:

```
SqlException: Invalid column name 'Rating'.
```

Цей виняток виникає через те, що оновлений клас моделі Movie відрізняється від схеми таблиці Movie у базі даних. У таблиці немає стовпця **Rating**.

Існує кілька способів усунення цієї помилки:

1. **Автоматично видалити й повторно створити базу даних** з використанням нової схеми класу моделі.
    - Цей підхід зручний на ранніх етапах розробки, оскільки дозволяє швидко розвивати модель і схему бази даних разом.
    - Недолік: усі наявні дані в базі даних буде втрачено.
    - **Не використовуйте цей підхід у production-базі даних!**

    - Видалення бази даних при зміні схеми та використання ініціалізатора для автоматичного наповнення бази тестовими даними часто є продуктивним способом розробки застосунку.
2. **Явно змінити схему існуючої бази даних**, щоб вона відповідала класам моделі.
    - Перевага цього підходу — збереження даних.
    - Зміни можна зробити вручну або створивши скрипт для зміни бази даних.
3. **Використати EF Core Migrations** для оновлення схеми бази даних.

У цьому навчальному посібнику використовується **EF Core Migrations**.
Додатково потрібно оновити клас **SeedData**, щоб він забезпечував значення для нового стовпця.
Нижче наведено приклад зміни, але її необхідно зробити для кожного блоку Movie: new Movie

```
context.Movie.AddRange(
    new Movie
    {
        Title = "When Harry Met Sally",
        ReleaseDate = DateTime.Parse("1989-2-12"),
        Genre = "Romantic Comedy",
        Price = 7.99M,
        Rating = "R"
    },
```

Створіть додаток – натисніть Ctrl+Shift+B

Додавання міграції для поля *Rating*

1. У меню Tools виберіть **NuGet Package Manager > Package Manager Console**.

У Package Manager Console (PMC) введіть наступну команду:

```
    Add-Migration Rating
```

Команда **Add-Migration** дає інструкцію фреймворку:
- Порівняти модель **Movie** зі схемою таблиці **Movie** у базі даних.
- Створити код для міграції схеми бази даних до нової моделі.

Назва "*Rating*" є довільною та використовується для іменування файлу міграції. Корисно надавати файлам міграцій осмислені назви.

2. У PMC введіть команду:

```
Update-Database
```

Команда **Update-Database** дає інструкцію фреймворку застосувати зміни до схеми бази даних та зберегти наявні дані.

Оновлення даних

Видаліть усі записи з бази даних — ініціалізатор (*initializer*) знову заповнить базу даних і включить поле Rating.

- Видалення можна зробити за допомогою посилань Delete у браузері або через Sql Server Object Explorer (SSOX).

Інший варіант — видалити базу даних і використати міграції для її повторного створення.

Щоб видалити базу даних у SSOX:

- Виберіть базу даних у SSOX.
- Клацніть правою кнопкою миші та виберіть Delete.
- Поставте прапорець Close existing connections (*Закрити наявні з'єднання*).
- Натисніть OK.

2. У PMC оновіть базу даних командою:

```
Update-Database
```

Перевірка

Запустіть застосунок і переконайтеся, що можна створювати, редагувати та відображати фільми з полем Rating.

Якщо база даних не була ініціалізована, встановіть точку зупину (*breakpoint*) у методі SeedData.Initialize.

Додавання валідації

У цьому розділі до моделі Movie додана логіка валідації. Правила перевірки застосовуються щоразу, коли користувач створює або редагує фільм.

Перевірки

Ключовий принцип розробки програмного забезпечення називається DRY (**D**on't **R**epeat **Y**ourself - «**Не повторюйся»). Razor Pages заохочує розробку, де функціональність** вказується один раз, і це відображається у всьому додатку. DRY може допомогти:

- Зменште кількість коду в додатку.
- Зробіть код менш схильним до помилок, а також простішим для тестування та підтримки.

Підтримка валідації, що надається Razor Pages і Entity Framework, є хорошим прикладом принципу DRY:

- Правила валідації декларативно задаються в одному місці, в класі моделі.
- Правила застосовуються скрізь у додатку.

Додавання правил перевірки до моделі фільму

Простір імен System.ComponentModel.DataAnnotations забезпечує:

• Набір вбудованих атрибутів валідації, які застосовуються декларативно до класу або властивості.

• Атрибути форматування, як-от `[DataType]`, які допомагають із форматуванням і не забезпечують жодної перевірки.

Оновіть клас `Movie`, щоб скористатися перевагами вбудованих атрибутів перевірки **[Required], [StringLength], [RegularExpression]** та **[Range]**.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace RazorPagesMovie.Models;

public class Movie
{
    public int Id { get; set; }

    [StringLength(60, MinimumLength = 3)]
    [Required]
    public string Title { get; set; } = string.Empty;

    [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }

    [Range(1, 100)]
    [DataType(DataType.Currency)]
    [Column(TypeName = "decimal(18, 2)")]
    public decimal Price { get; set; }

    [RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$")]
    [Required]
    [StringLength(30)]
    public string Genre { get; set; } = string.Empty;

    [RegularExpression(@"^[A-Z]+[a-zA-Z0-9""'\s-]*$")]
    [StringLength(5)]
    [Required]
    public string Rating { get; set; } = string.Empty;
}
```

Атрибути валідації визначають поведінку, яку необхідно застосовувати до властивостей моделі, до яких вони додані:

- Атрибути **[Required]** та **[MinimumLength]** вказують, що властивість обов'язково повинна мати значення.
  Нічого не заважає користувачеві ввести лише пробіли, щоб задовольнити цю перевірку.
- Атрибут **[RegularExpression]** використовується для обмеження символів, які можна вводити.
  У наведеному вище коді поле **Genre**:
  - повинно містити лише літери,
  - перша літера має бути великою,
  - дозволені пробіли, але **числа та спеціальні символи не допускаються**.

Для поля **Rating** регулярний вираз:

- вимагає, щоб перший символ був великою літерою,

    - дозволяє спеціальні символи та числа в наступних позиціях. Наприклад, "*PG-13*" є коректним значенням для **Rating**, але не підходить для **Genre**.
- Атрибут **[Range]** обмежує значення в межах указаного діапазону.
- Атрибут **[StringLength]** може встановлювати максимальну довжину рядкової властивості та за потреби її мінімальну довжину.

Типи значень, такі як decimal, int, float, DateTime, за своєю природою є обов'язковими й не потребують атрибуту **[Required]**.

Наведені правила валідації використовуються для демонстрації, вони не є оптимальними для production-системи.
Наприклад, вони:

- забороняють введення фільму з назвою лише з двох символів,
- не дозволяють спеціальні символи у полі **Genre**.

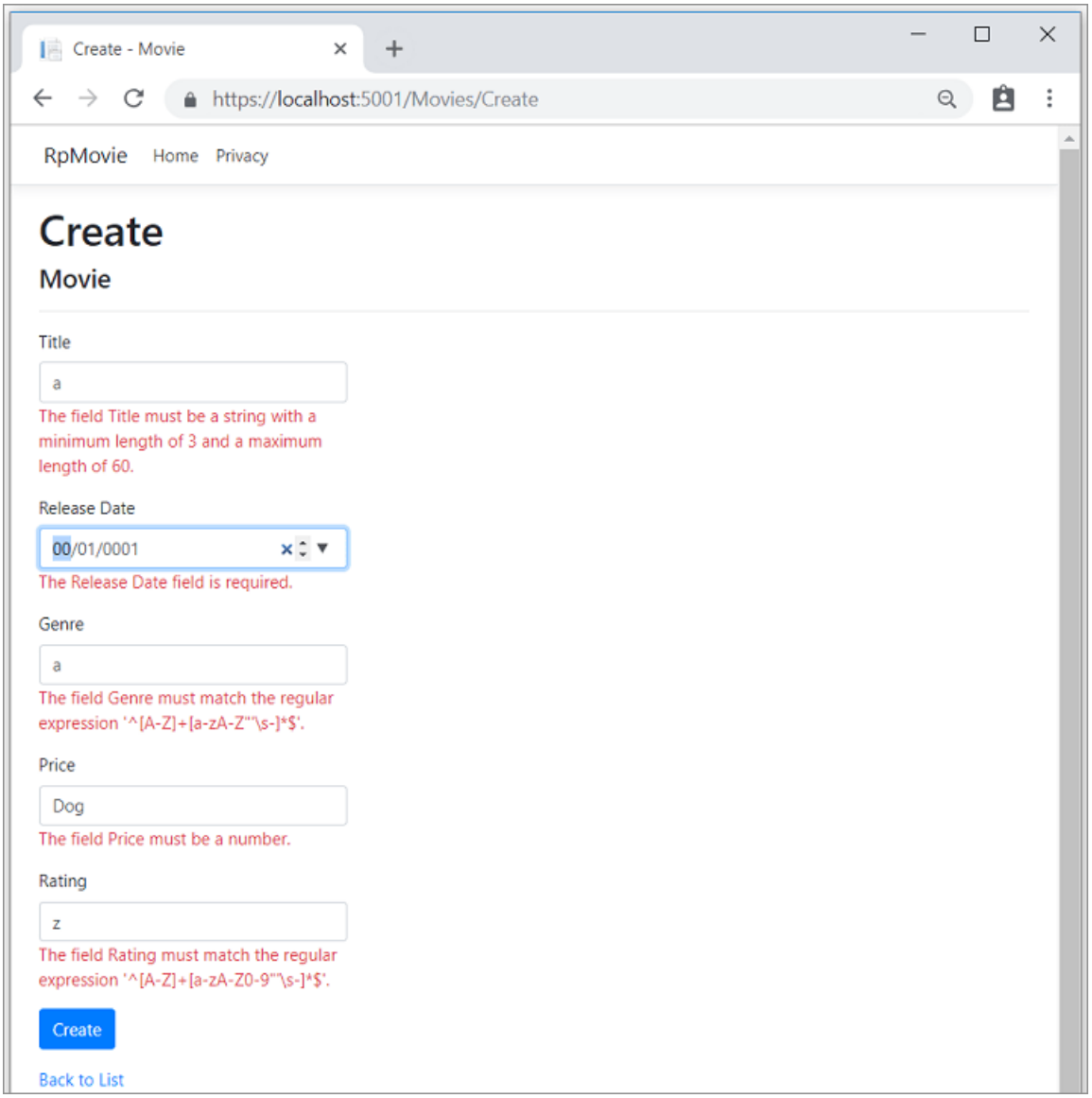
Автоматичне застосування правил валідації в ASP.NET Core допомагає:

- зробити застосунок більш надійним,
- зменшити ймовірність збереження некоректних даних у базу.

Інтерфейс повідомлень про помилки валідації у Razor Pages

1. Запустіть застосунок і перейдіть на сторінку **Pages/Movies**.
2. Виберіть посилання **Create New**.
3. Заповніть форму з деякими некоректними значеннями.

Коли **jQuery client-side validation** виявляє помилку, воно відображає повідомлення про помилку.



Примітка

Можливо, ви не зможете вводити десяткові коми у числові поля. Щоб підтримати валідацію jQuery для неанглійських локалей, де кома (,) використовується як десятковий роздільник, а також для форматів дат, відмінних від US-English, потрібно виконати глобалізацію застосунку. Інструкції щодо додавання підтримки десяткової коми див. у цьому коментарі GitHub #4076.

Зверніть увагу, що форма автоматично відобразила повідомлення про помилку в кожному полі з некоректним значенням.

Помилки перевіряються як на клієнті (за допомогою JavaScript та jQuery), так і на сервері (коли у користувача вимкнено JavaScript).

Важливою перевагою є те, що не потрібно було змінювати код на сторінках **Create** або **Edit**.

Щойно до моделі були застосовані анотації даних (*data annotations*), UI валідації було активовано.

Сторінки **Razor Pages**, створені в цьому посібнику, автоматично врахували правила

валідації, використовуючи атрибути валідації у властивостях класу моделі Movie. Перевірте валідацію на сторінці Edit — діятимуть ті самі правила.

Поведінка даних форми

Дані з форми не надсилаються на сервер, доки існують клієнтські помилки валідації.

Переконатися в цьому можна кількома способами:

- Встановіть точку зупину у методі **OnPostAsync**.

Натисніть **Create** або **Save** для відправлення форми. Точка зупину не спрацює.

- Використайте утиліту **Fiddler**.
- Використайте інструменти розробника браузера для моніторингу мережевого трафіку.

Серверна валідація

Коли JavaScript у браузері вимкнено, відправлення форми з помилками все одно передасть дані на сервер.

Додатково: перевірка серверної валідації

1. Вимкніть JavaScript у браузері. (Це можна зробити через інструменти розробника. Якщо у браузері це неможливо, спробуйте інший.)
2. Встановіть точку зупину в методі **OnPostAsync** на сторінці **Create** або Edit.
3. Відправте форму з некоректними даними.
4. Переконайтеся, що стан моделі (**ModelState**) є невалідним.

```
if (!ModelState.IsValid)
 {
    return Page();
 }
```

Або вимкніть перевірку на стороні клієнта на сервері.

Наступний код показує частину каркасу сторінки `Create.cshtml`, створеної раніше в навчальному посібнику. Він використовується на сторінках «Create » та «Edit » для:

- Відображення початкової форми.
- Повторного відображення форми в разі помилки.

CSHTML

```
<form method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="form-group">
        <label asp-for="Movie.Title" class="control-label"></label>
        <input asp-for="Movie.Title" class="form-control" />
        <span asp-validation-for="Movie.Title" class="text-danger"></span>
    </div>
```

Input Tag Helper використовує атрибути **DataAnnotations** і створює HTML-атрибути, необхідні для валідації на клієнтській стороні за допомогою jQuery Validation.
Validation Tag Helper відображає повідомлення про помилки валідації.
Докладніше див. у розділі *Validation*.

Сторінки **Create** та **Edit** не містять власних правил валідації.
Усі правила валідації та текст помилок вказуються лише в класі **Movie**.
Ці правила автоматично застосовуються до Razor Pages, які редагують модель **Movie**.
Якщо потрібно змінити логіку валідації, це робиться тільки в моделі.
Валідація застосовується послідовно у всьому застосунку, а логіка валідації визначена в одному місці.
Такий підхід допомагає підтримувати код чистим, простішим у супроводі та оновленні.

Використання атрибутів DataType

Розглянемо клас **Movie**. Простір імен System.ComponentModel.DataAnnotations надає форматувальні атрибути, крім вбудованого набору атрибутів валідації.
Атрибут **[DataType]** застосовується до властивостей **ReleaseDate** та **Price**.

```
[DataType(DataType.Date)]
public DateTime ReleaseDate { get; set; }

[Range(1, 100)]
[DataType(DataType.Currency)]
[Column(TypeName = "decimal(18, 2)")]
public decimal Price { get; set; }
```

Атрибути **[DataType]** забезпечують:

- підказки для рушія подань щодо форматування даних;
- генерацію HTML-атрибутів, наприклад <a> для URL-адрес і <a href="mailto:EmailAddress.com"> для електронної пошти.

Атрибут **[RegularExpression]** використовується для перевірки формату даних.
Атрибут **[DataType]** вказує тип даних, який є більш специфічним, ніж вбудований тип бази даних.
[DataType] не є атрибутом валідації.
У прикладі додатка відображається лише дата, без часу.
Перелік DataType надає багато типів: Date, Time, PhoneNumber, Currency, EmailAddress та інші.

Атрибути DataType:

- можуть автоматично вмикати специфічні функції у застосунку (наприклад, створення посилання *mailto:* для **DataType.EmailAddress**);
- можуть надавати селектор дати (**DataType.Date**) у браузерах, що підтримують HTML5;
- створюють HTML5-атрибути **data-**, які обробляються браузерами;
- **не надають валідації**.

DataType.Date не визначає формат відображення дати.
За замовчуванням поле дати відображається згідно з форматами, визначеними параметром **CultureInfo** сервера.
Атрибут [Column(TypeName = "decimal(18, 2)")] потрібен для того, щоб **Entity Framework Core** коректно відобразив Price у валютному форматі в базі даних.
Докладніше див. у розділі *Data Types*.

Атрибут DisplayFormat
Атрибут **[DisplayFormat]** використовується для явного задання формату дати:

```
[DisplayFormat(DataFormatString = "{0:yyyy-MM-dd}", ApplyFormatInEditMode = true)]
public DateTime ReleaseDate { get; set; }
```

Властивість ApplyFormatInEditMode вказує, що форматування буде застосоване при редагуванні значення.
Це може бути небажаним для деяких полів.
Наприклад, для валютних значень зазвичай не потрібно показувати символ валюти в UI редагування.
Атрибут **[DisplayFormat]** може використовуватися сам по собі, але зазвичай рекомендується комбінувати його з **[DataType]**.
Атрибут **[DataType]** передає семантику даних, а не спосіб відображення на екрані.

Переваги [DataType] порівняно з [DisplayFormat]:

- браузер може вмикати функції HTML5 (календар, локалізований символ валюти, email-посилання тощо);
- браузер за замовчуванням відображає дані у правильному форматі відповідно до своєї локалі;
- ASP.NET Core може обрати правильний шаблон відображення даних.
  Якщо використовується тільки **DisplayFormat**, застосовується рядковий шаблон.

Примітка
jQuery validation не працює з атрибутом **[Range]** і DateTime.
Наприклад, наступний код завжди показуватиме помилку валідації на клієнті, навіть якщо дата входить у заданий діапазон:

```
[Range(typeof(DateTime), "1/1/1966", "1/1/2020")]
```

Рекомендовано уникати «жорсткого» вбудовування дат у моделі.
Замість цього використовуйте **Configuration** для визначення діапазонів дат та інших значень, що можуть часто змінюватися.
Наступний код показує об'єднання атрибутів в один рядок:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace RazorPagesMovie.Models;

public class Movie
{
    public int Id { get; set; }

    [StringLength(60, MinimumLength = 3)]
    public string Title { get; set; } = string.Empty;

    [Display(Name = "Release Date"), DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }

    [RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$"), Required, StringLength(30)]
    public string Genre { get; set; } = string.Empty;

    [Range(1, 100), DataType(DataType.Currency)]
    [Column(TypeName = "decimal(18, 2)")]
    public decimal Price { get; set; }
```

```
    [RegularExpression(@"^[A-Z]+[a-zA-Z0-9""'\s-]*$"), StringLength(5)]
    public string Rating { get; set; } = string.Empty;
}
```

Початок роботи з Razor Pages та EF Core демонструє розширені операції EF Core із Razor Pages: https://learn.microsoft.com/uk-ua/aspnet/core/data/ef-rp/intro?view=aspnetcore-9.0

Застосування міграцій

Атрибути **DataAnnotations**, застосовані до класу, змінюють схему бази даних. Наприклад, атрибути, застосовані до поля **Title**:

```
[StringLength(60, MinimumLength = 3)]
[Required]
public string Title { get; set; } = string.Empty;
```

- обмежують довжину рядка 60 символами;
- забороняють значення null.

Таблиця **Movie** наразі має таку схему:

SQL

```
CREATE TABLE [dbo].[Movie] (
    [ID]          INT             IDENTITY (1, 1) NOT NULL,
    [Title]       NVARCHAR (MAX)  NULL,
    [ReleaseDate] DATETIME2 (7)   NOT NULL,
    [Genre]       NVARCHAR (MAX)  NULL,
    [Price]       DECIMAL (18, 2) NOT NULL,
    [Rating]      NVARCHAR (MAX)  NULL,
    CONSTRAINT [PK_Movie] PRIMARY KEY CLUSTERED ([ID] ASC)
);
```

Попередні зміни схеми не призводять до того, що EF генерує виняток. Однак створіть міграцію, щоб схема відповідала моделі.

У меню **«Інструменти»** виберіть **NuGet Package Manager > Package Manager Console**. У PMC введіть такі команди:

PowerShell

```
Add-Migration New_DataAnnotations
Update-Database
```

`Update-Database` запускає метод `Up` класу `New_DataAnnotations`.
Перегляньте метод `Up`:

```
public partial class New_DataAnnotations : Migration
{
    /// <inheritdoc />
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AlterColumn<string>(
            name: "Title",
            table: "Movie",
            type: "nvarchar(60)",
            maxLength: 60,
```

```
            nullable: false,
            oldClrType: typeof(string),
            oldType: "nvarchar(max)");

        migrationBuilder.AlterColumn<string>(
            name: "Rating",
            table: "Movie",
            type: "nvarchar(5)",
            maxLength: 5,
            nullable: false,
            oldClrType: typeof(string),
            oldType: "nvarchar(max)");

        migrationBuilder.AlterColumn<string>(
            name: "Genre",
            table: "Movie",
            type: "nvarchar(30)",
            maxLength: 30,
            nullable: false,
            oldClrType: typeof(string),
            oldType: "nvarchar(max)");
    }
```

Оновлена таблиця «**Movies**» має наступну схему:

SQL

```
CREATE TABLE [dbo].[Movie] (
    [ID]          INT             IDENTITY (1, 1) NOT NULL,
    [Title]       NVARCHAR (60)   NOT NULL,
    [ReleaseDate] DATETIME2 (7)   NOT NULL,
    [Genre]       NVARCHAR (30)   NOT NULL,
    [Price]       DECIMAL (18, 2) NOT NULL,
    [Rating]      NVARCHAR (5)    NOT NULL,
    CONSTRAINT [PK_Movie] PRIMARY KEY CLUSTERED ([ID] ASC)
);
```